

Flight Dynamics — Concept

Explanation:

How Motor Speed Controls Movement:

A quadcopter has four motors arranged in an X configuration. Two motors (1 and 3, or 2 and 4 depending on layout) spin clockwise (CW) and the other two spin counter-clockwise (CCW). This opposite rotation is essential — it cancels out the spinning torque each motor creates, so the drone doesn't just spin in place. All four motors are controlled independently by the flight controller, which sends signals through Electronic Speed Controllers (ESCs) to adjust each motor's RPM individually.

Movement is created entirely by varying the relative speed between motors:

Throttle (Up/Down): All four motors increase speed equally to climb, or decrease equally to descend. This is the only control that uses all motors uniformly.

Pitch (Forward/Backward): The drone tilts along its lateral axis. To move forward, the rear motors spin faster than the front motors, tilting the nose down so thrust pushes the drone forward. To move backward, the front motors speed up instead.

Roll (Left/Right): The drone tilts along its longitudinal axis. To roll left, the right-side motors spin faster than the left-side motors, tilting the drone left so thrust pushes it sideways.

Rolling right works the opposite way.

Yaw (Rotate Left/Right): This uses the difference in torque between CW and CCW motor pairs. Speeding up the two CW motors while slowing the two CCW motors (or vice versa) creates a net rotational torque, spinning the drone around its vertical axis without changing its position.

In every case, the drone doesn't "steer" like a car — it tilts its entire body in the direction it wants to move, and gravity plus thrust vector combine to create directional motion.

How the Drone Maintains Balance

Balance (stabilization) relies on continuous sensor feedback and rapid motor correction, working as follows:

Sensing: An Inertial Measurement Unit (IMU), containing a gyroscope and accelerometer, constantly measures the drone's current angle and rotational rate on all three axes (roll, pitch, yaw).

Comparison: The flight controller compares this actual orientation against the desired orientation (set by the pilot's stick input or an autonomous flight plan).

Correction: Any difference between actual and desired orientation is called "error." The flight controller calculates how much to adjust each motor's speed to reduce this error — this is where PID control comes in (explained in Deliverable 2).

Motor Mixing: The correction values for roll, pitch, yaw, and throttle are combined ("mixed") into four individual motor speed commands, since each motor contributes to multiple axes simultaneously.

Continuous Loop: This entire sense-compare-correct cycle repeats hundreds of times per second, which is why a quadcopter can recover from a gust of wind or a bump almost instantly — it's constantly self-correcting in real time rather than holding a fixed position.

This closed-loop feedback system is what allows a drone to hover steadily in place even without any pilot input, and it's the same underlying mechanism used in real flight controllers like the KK 2.1.5, PX4, and ArduPilot.

PID Control

What is PID Control?

PID stands for Proportional, Integral, Derivative — three mathematical terms that work together to calculate exactly how much correction a system needs based on its current "error" (the difference between the desired value and the actual value). It is the core stabilization method used in real flight controllers, including PX4 and ArduPilot.

The Three Components

P (Proportional): Responds to the current error. The bigger the error, the bigger the correction. If the drone tilts far from level, P applies a strong correction immediately. However, P alone usually leaves a small permanent offset called steady-state error, because the correction shrinks to near-zero as the error shrinks, never fully closing the gap.

I (Integral): Responds to accumulated error over time. It keeps track of how long and how much the error has persisted, and slowly adds extra correction to eliminate that leftover offset P couldn't fix. This is why a PI controller settles much closer to the actual target than P alone. The tradeoff is that too much I can cause overshoot and sluggish settling.

D (Derivative): Responds to how fast the error is changing. It predicts where the system is heading and applies a "braking" effect to prevent overshoot — essentially smoothing out the response so the drone doesn't oscillate back and forth before settling. Adding D generally improves stability when tuned correctly.

Combined, the controller calculates a correction signal:

$$\text{Output} = (P \times \text{error}) + (I \times \text{accumulated error}) + (D \times \text{rate of change of error})$$

How PID Stabilizes the Drone

The flight controller reads the desired angle (from the pilot or mission plan) and the actual angle (from the gyroscope/IMU). It calculates the error (difference between the two).

The PID formula converts that error into a correction value for each axis (roll, pitch, yaw).

This correction is mixed with throttle input and sent individually to each motor through the ESCs.

The motors adjust speed accordingly, the drone's orientation changes, and the IMU reads the new actual angle — restarting the loop.

Many real systems (including ArduPilot, referenced in the PDF) use a cascaded PID structure: an outer "Stabilize PID" loop controls angle, and feeds its output as a target rotation rate into an inner "Rate PID" loop, which directly controls motor output.

This two-layer approach gives both accurate positioning and fast, smooth response.

Why Tuning Matters

Increasing each gain has tradeoffs:

Higher K_p → faster response, but more overshoot and reduced stability if too high.

Higher K_i → eliminates steady-state error, but increases settling time and overshoot.

Higher K_d → reduces overshoot and improves stability, but is very sensitive to sensor noise.

Well-tuned PID values let a drone hover rock-steady, resist wind disturbance, and follow waypoints smoothly — which is exactly why every modern flight controller (KK 2.1.5, PX4, ArduPilot) relies on it as its core stability system.